

Memória de Decisão e Justificativa das Mudanças

A. Critério de consolidação usado

Critério aplicado (olympus-plan-auditor):

1. Prioridade para coerência arquitetural e executabilidade.
2. Precedência fixa: 2026 > GitFlow válido > 2025 reaproveitável.
3. Rejeição explícita de conflito não resolvido.
4. Promoção de regra somente quando suportada por base documental + auditoria.
5. Fechamento de lacunas exigidas pelo próprio 2026 (ADRs/specs/tasks).

B. Documento que passa a ser canônico

Documento canônico vigente:

- “Consolidado Canônico do Projeto Olympus Climb (Base Vigente)” (Documento 1 desta entrega).

Papel dos originais após consolidação:

- `OLYMPUS CLIMB 2026`: referência-mãe de visão e arquitetura.
- `GitFlow`: referência histórica parcial de disciplina operacional.
- `GAMEPLAN 2025`: legado técnico para stack/checklists de infra.

C. Elementos mantidos sem mudança

Mantidos integralmente:

1. Direção agent-centric.
2. Organização por capacidades.
3. Camadas macro da arquitetura.
4. Human approval para ações sensíveis.
5. Observabilidade e audit trail como obrigação.
6. Evals como requisito de confiabilidade.
7. Regras de rastreabilidade de decisão.

Razão:

- coerência interna alta no 2026 e alinhamento direto com objetivo do produto.

D. Elementos mantidos com adaptação

1. GitFlow disciplinar mantido, mas adaptado:

- Mantido: `main` protegida, PR, CI gate, revisão, SemVer.
- Adaptado: branch por capability/agent/skill/eval em vez de fase.
- Conflito resolvido: cronograma-fase vs arquitetura evolutiva.

2. Stack do 2025 mantida, mas reposicionada:

- Mantido: Postgres/Prisma/Redis/BullMQ/PM2/Expo/NativeWind/Jest/Artillery.
- Adaptado: stack vira meio de execução, não identidade do produto.
- Conflito resolvido: scraper/app-centric vs decision-centric.

3. Meta de documentação mantida, mas com gate:

- TypeDoc mantido como alvo.
- Adaptado: só após boundaries/contratos/TSDoc.
- Lacuna resolvida: documentação viva sem base estável.

E. Elementos descartados

Descartes explícitos:

1. Execução por fases como eixo principal.
2. Branch naming `feature/fase-X`.
3. `scraper-core-dev` como pilar estratégico.
4. Tratamento de scraper/dashboard/mobile como centros independentes.
5. Leitura dos três PDFs como equivalentes de autoridade.

Por quê:

- conflitam com arquitetura canônica 2026;
- distorcem planejamento para cronologia, não para capacidade;
- aumentam risco de retrabalho estrutural.

F. Conflitos resolvidos

1. Faseamento vs capability-driven:

- resolução: capability-driven obrigatório para planejamento e branching.

2. Scraper-first vs agentic system:

- resolução: scraper rebaixado a subcapacidade de descoberta/execução.

3. GitFlow antigo vs branch por capability/agent/skill:

- resolução: disciplina Git mantida, naming e eixo de branch substituídos.

4. Ausência de ADRs:

- resolução: ADRs mínimas definidas como gate de prontidão.

5. Ausência de specs executáveis:

- resolução: catálogo mínimo de specs obrigatório antes de implementação pesada.

6. Ausência de task manifests:

- resolução: task manifest obrigatório para qualquer spec executável.

G. Lacunas preenchidas

Lacunas que o consolidado fecha:

1. Definição oficial do que está vigente.
2. Regra de precedência documental.
3. Política Git compatível com arquitetura 2026.
4. Gates de prontidão (ADRs/specs/tasks).
5. Papel formal do TypeDoc e momento correto de adoção.
6. Passagem de `doc/` histórico para `docs/` como fonte viva.

H. Impacto das mudanças na execução do projeto

Impactos positivos diretos:

1. Reduz ambiguidade de decisão entre documentos concorrentes.
2. Permite onboarding de colaborador com trilha clara.
3. Diminui risco de engenharia ad hoc.
4. Cria base para delegação segura entre humanos/agentes.
5. Converte visão estratégica em governança executável.

Trade-off explícito:

- aumento de esforço inicial documental antes de construir feature.
- ganho líquido: menor retrabalho, menor acoplamento, maior previsibilidade.

I. Riscos que ainda permanecem

1. Risco de overengineering agentic se tentar abrir muitas frentes simultâneas.
2. Risco de under-spec em contratos críticos se specs forem superficiais.
3. Risco de custo operacional alto sem política de budget por execução.
4. Risco de observabilidade virar “logging cosmético” sem métricas acionáveis.
5. Risco de atraso se ADR/spec/task não tiverem owner e prazo explícitos.

J. Próximas formalizações obrigatórias (ADRs/specs/task manifests)

Sequência obrigatória imediata:

1. Publicar ADR-001..ADR-008 (mínimas do consolidado).
2. Publicar SPEC-001..SPEC-009 (mínimas do consolidado).
3. Gerar task manifests para o primeiro ciclo vertical:
 - descoberta;
 - normalização;
 - match/strategy;
 - approval;
 - evidência.

Critério de transição:

- sem esses artefatos aceitos, o projeto permanece “não pronto” para implementação funcional pesada.

Fechamento curto

O que este consolidado já resolve

- Define uma única fonte de verdade vigente.
- Elimina conflito de autoridade entre 2026, GitFlow e 2025.
- Fecha política arquitetural, operacional e de governança mínima.
- Define gates objetivos de prontidão para começar a implementar com segurança.

O que ainda precisa ser formalizado em documento separado

- ADRs aceitas (com status e histórico de revisão).
- Specs executáveis completas por capacidade.
- Task manifests da primeira vertical ponta-a-ponta.
- Convenção TSDoc e política de geração/publicação do TypeDoc.
- Runbooks operacionais de validação e incidentes.